

bok25

基
礎
班

、**考研数据結構**



线性表的查找

顺序查找

折半查找

分块查找



查找概念介绍

查找表：由同一类型的数据元素(或记录)构成的集合 (待查找的一大堆数据)

关键字：数据元素(或记录)中某个数据项的值，用它可以标识一个数据元素 (查找的对象)

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

$$\sum_{i=1}^n p_i c_i$$

其中， P_i 为查找表中第*i*个记录的概率 $\sum_{i=1}^n p_i=1$, C_i 为找到表中其关键字与给定值相等的第*i*个记录所需进行的对比次数



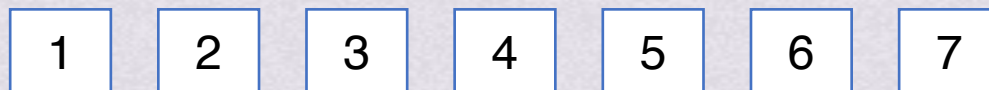
查找概念介绍

$$\sum_{i=1}^n p_i c_i$$

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

其中， P_i 为查找表中第*i*个记录的概率 $\sum_{i=1}^n p_i=1$, C_i 为找到表中其关键字与给定值相等的第*i*个记录时，和给定值已进行过比较的关键字个数

求：在以下数组中顺序查找成功的平均查找长度



查找成功的情况下，查找的数据必然存在于查找表中
即查找1,2,3...7的概率均为 $\frac{1}{7}$

$$\sum_{i=1}^n p_i c_i = \frac{1}{7} (1+2+3+4+5+6+7)$$

在每一个位置查找成功的概率

该位置的比较次数



顺序查找

折半查找

分块查找



顺序查找

从表的一端逐个比较到另一端

```
typedef struct{
    KeyType key;    //关键字域
    InfoType otherinfo; //其他域
}ElemType;
```

数据元素类型定义

```
typedef struct{
    ElemType *R; //存储空间基地址
    int length; //当前长度
}SSTable;
```

顺序表的定义

```
int Search_Seq(SSTable ST,KeyType key)
{ // 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素。
  //若找到，则函数值为该元素 在表中的位置，否则为 0
  for(i=ST.length;i>=1;--i)
    if(ST.R[i].key==key) return i; //从后往前查找
  return 0;
}
```

算法描述



顺序查找

从表的一端逐个比较到另一端

```
int Search_Seq(SSTable ST,KeyType key)
{
    // 在顺序表 ST 中顺序查找其关键字等于 key 的数据元素。
    // 若找到，则函数值为该元素在表中的位置，否则为0
    ST.R[0].key=key; //“监视哨”
    for(i=ST.length; ST.R[i].key!=key; --i); //从后往前找
    return i;
}
```

时间复杂度： $O(n)$

优点：算法简单，对表结构无任何要求，适用于顺序和链式存储

缺点：平均查找长度较大，查找效率较低，所以当n很大时，不宜采用顺序查找



折半查找（二分查找）

使用要求：1.线性表必须用顺序存储
2.线性表必须有序排列



折半查找（二分查找）

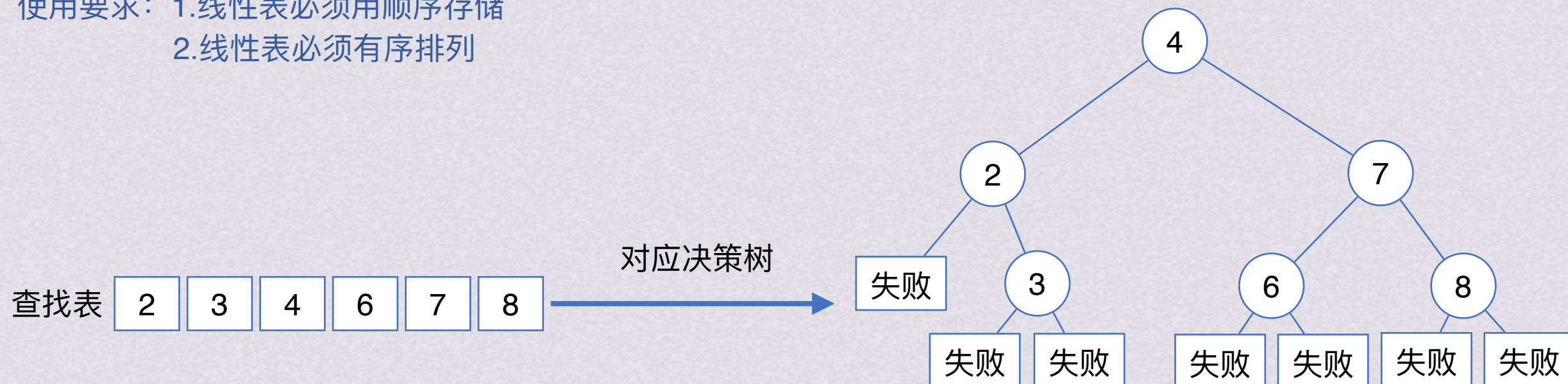
使用要求：1.线性表必须用顺序存储
2.线性表必须有序排列

```
int Search_Bin(SSTable ST,KeyType key)
{
    // 在有序表 ST 中折半查找其关键字等于 key 的数据元素。
    // 若找到，则函数值为该元素在表中的位置，否则为0
    low=1;high=ST.length;//置查找区间初值
    while(low<=high)
    {
        mid=(low+high)/2;
        if(key==ST.R[mid].key) return mid;    //找到待查元素
        else if(key<ST.R[mid].key) high=mid-1;//继续在前一子表进行查找
        else low=mid+1;//继续在后一子表进行查找
    }//while
    return 0;//表中不存在待查元素
}
```



折半查找（二分查找）

使用要求：1.线性表必须用顺序存储
2.线性表必须有序排列



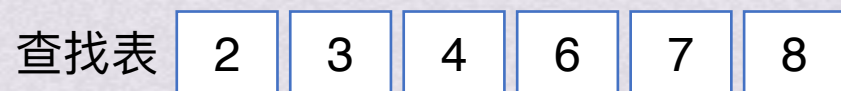
查找成功时的查找长度为从根节点到目的节点路径上的节点数
查找失败的查找长度为从根节点到对应失败节点的父节点路径上的节点数

折半查找时间复杂度为 $O(\log_2 n)$

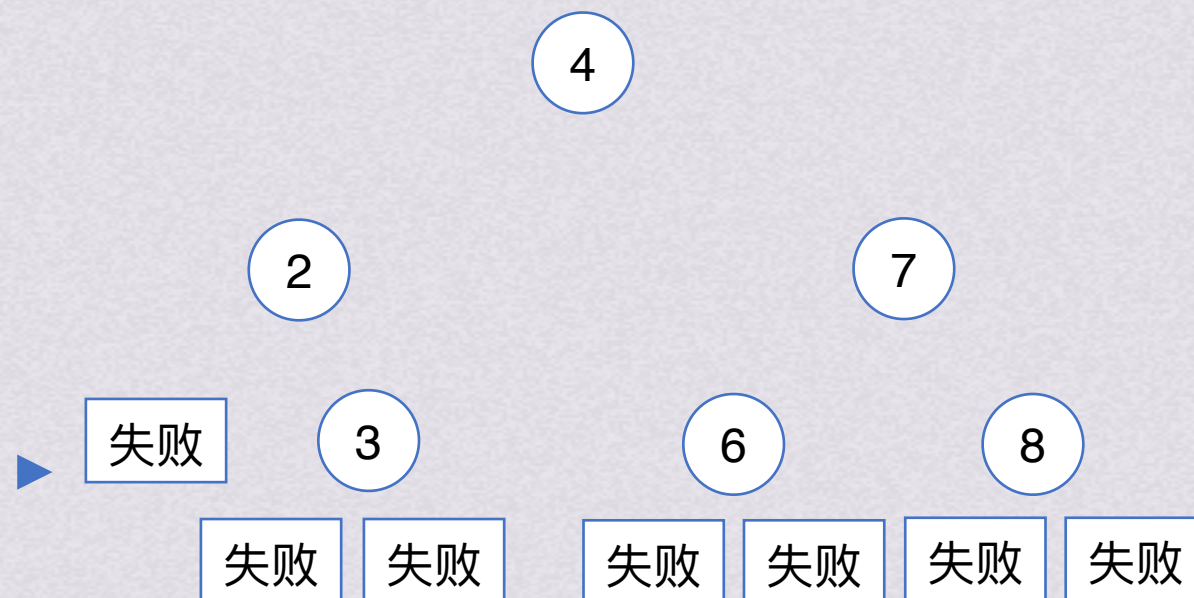


折半查找（二分查找）

使用要求：1.线性表必须用顺序存储
2.线性表必须有序排列



对应决策树



查找成功时的查找长度为从根节点到目的节点路径上的节点数
查找失败的查找长度为从根节点到对应失败节点的父节点路径上的节点数

$$ASL_{成功} = (1+2+2+3+3+3) / 6 = \frac{14}{6} = \frac{7}{3}$$

$$ASL_{失败} = (2+3+3+3+3+3+3) / 7 = \frac{20}{7}$$



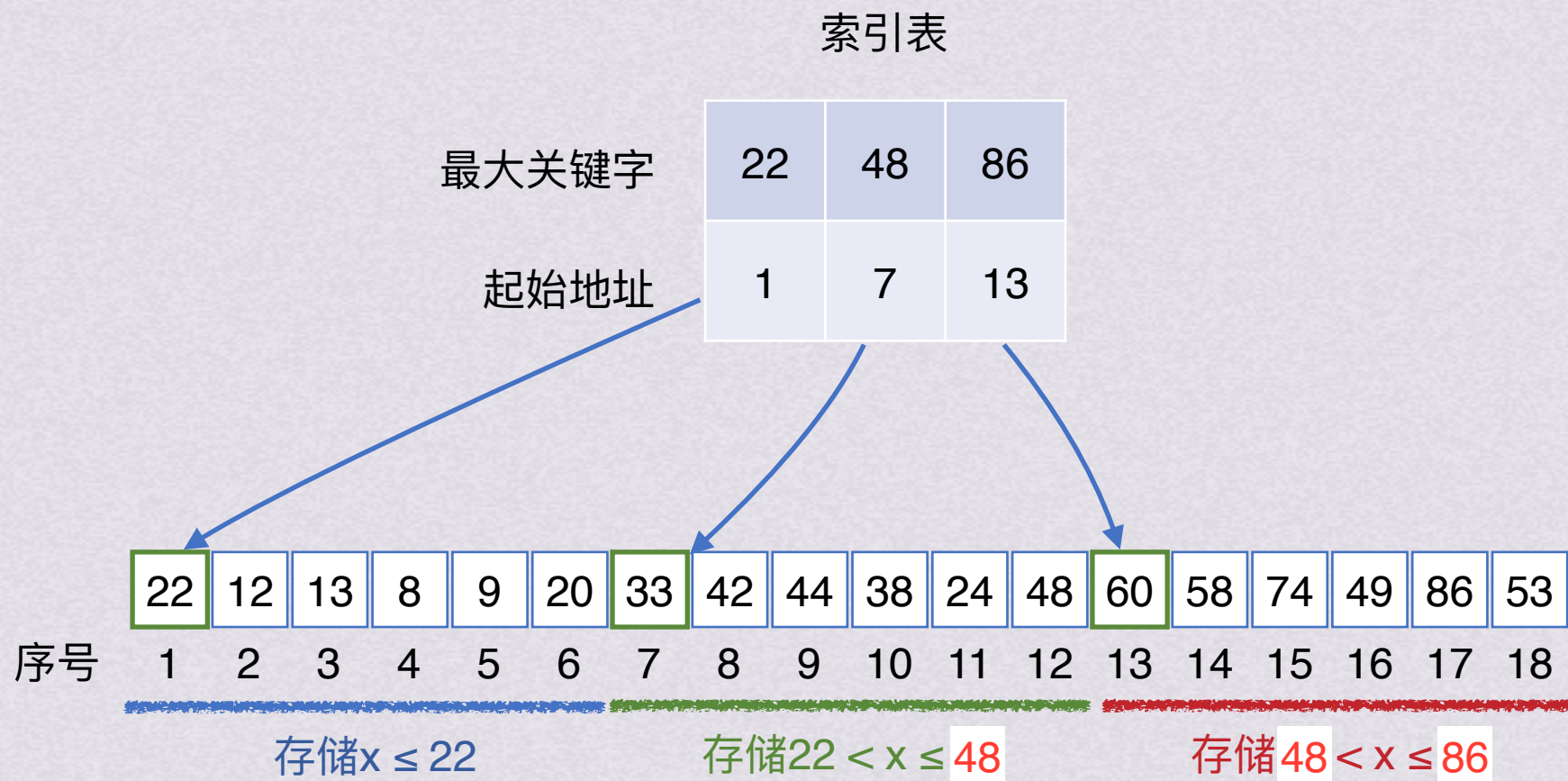
分块查找 (索引顺序查找)



分块查找

(索引顺序查找)

假设此时查找key=38





分块查找

(索引顺序查找)

假设此时查找key=38

1. 对比最大关键字

$22 < \text{key} < 48$, 从起始地址7开始找, 到12截止

索引表

最大关键字

22	48	86
1	7	13

起始地址





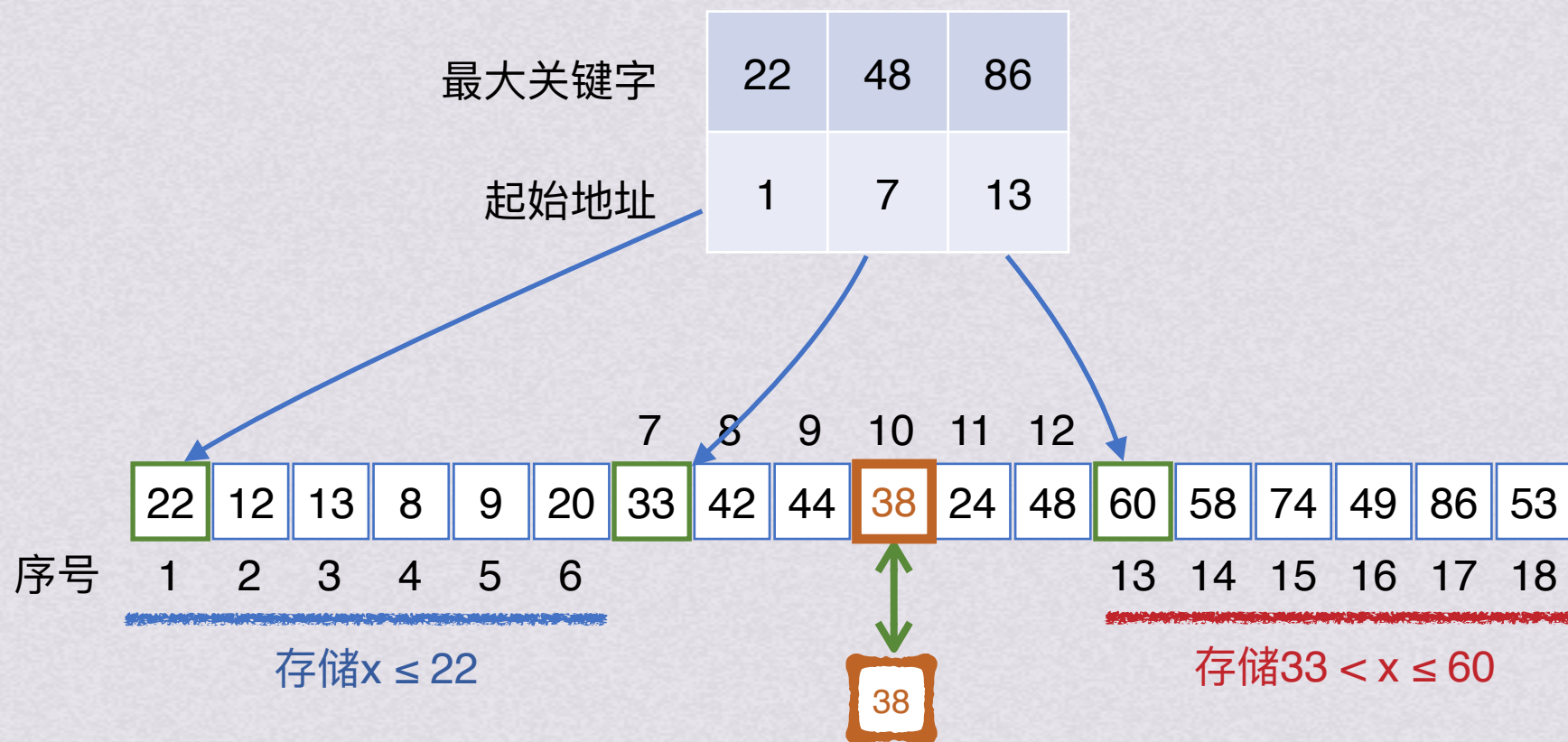
分块查找 (索引顺序查找)

假设此时查找key=38

1. 对比最大关键字

$22 < \text{key} < 48$ ，从起始地址7开始找，到12截止

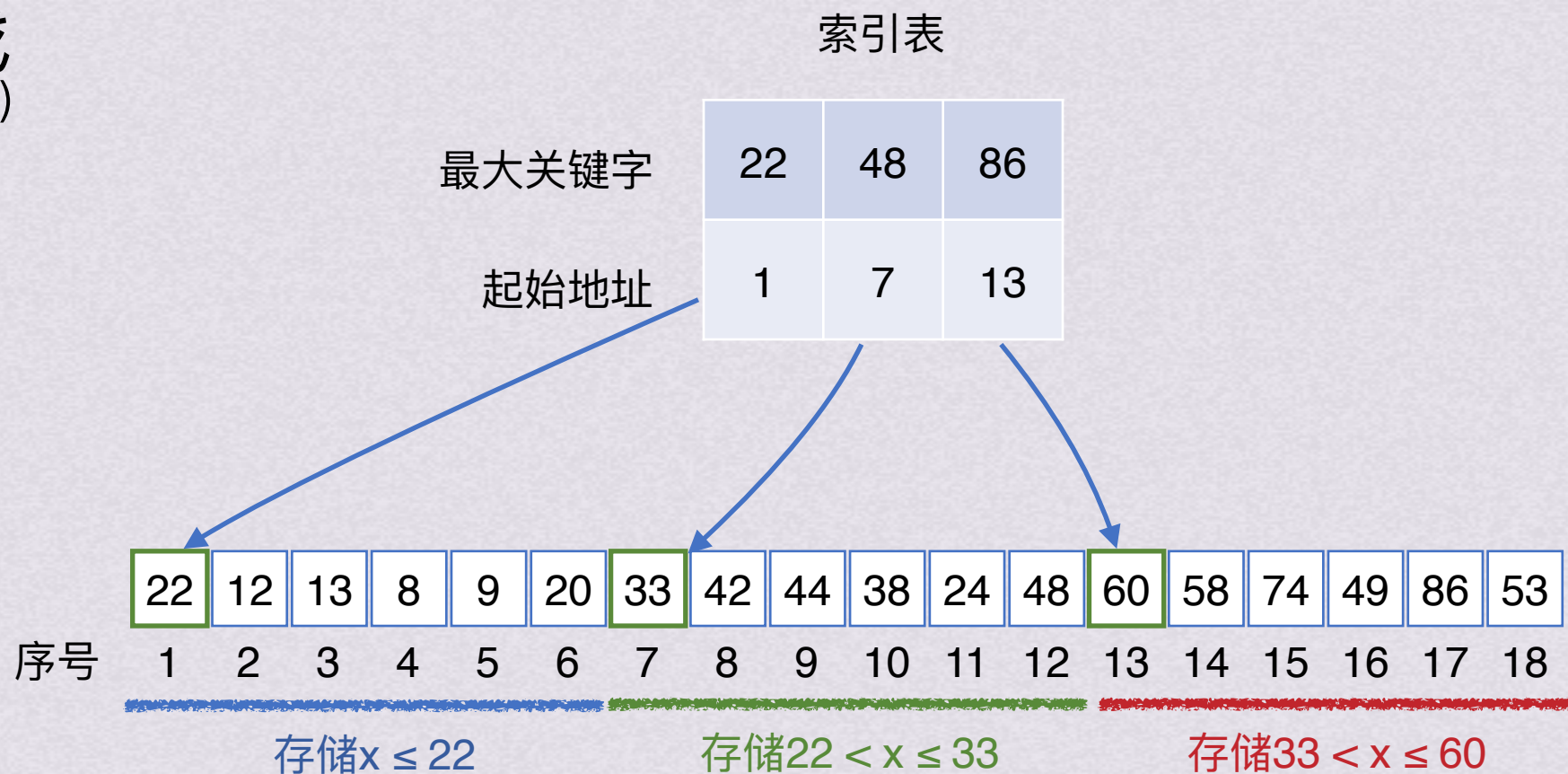
索引表





分块查找

(索引顺序查找)



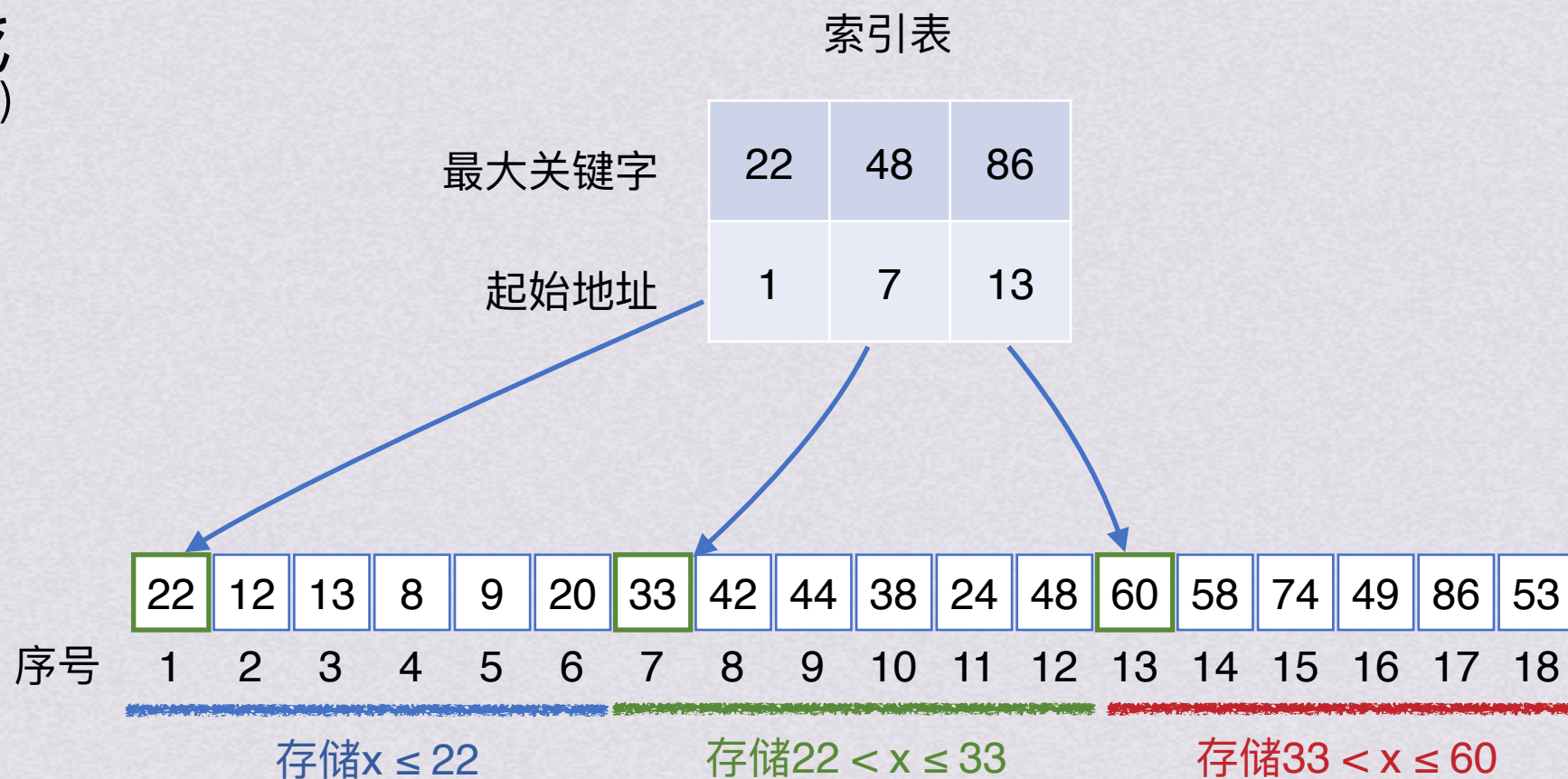
分块查找的平均查找长度为: $ASL_{bs} = L_b + L_w$

L_b : 查找索引表确定所在块的平均查找长度

L_w : 在块中查找元素的平均查找长度



分块查找 (索引顺序查找)



分块查找的平均查找长度为: $ASL_{bs} = L_b + L_w$

$$= \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2} \left(\frac{n}{s} + s \right) + 1$$

L_b : 查找索引表确定所在块的平均查找长度

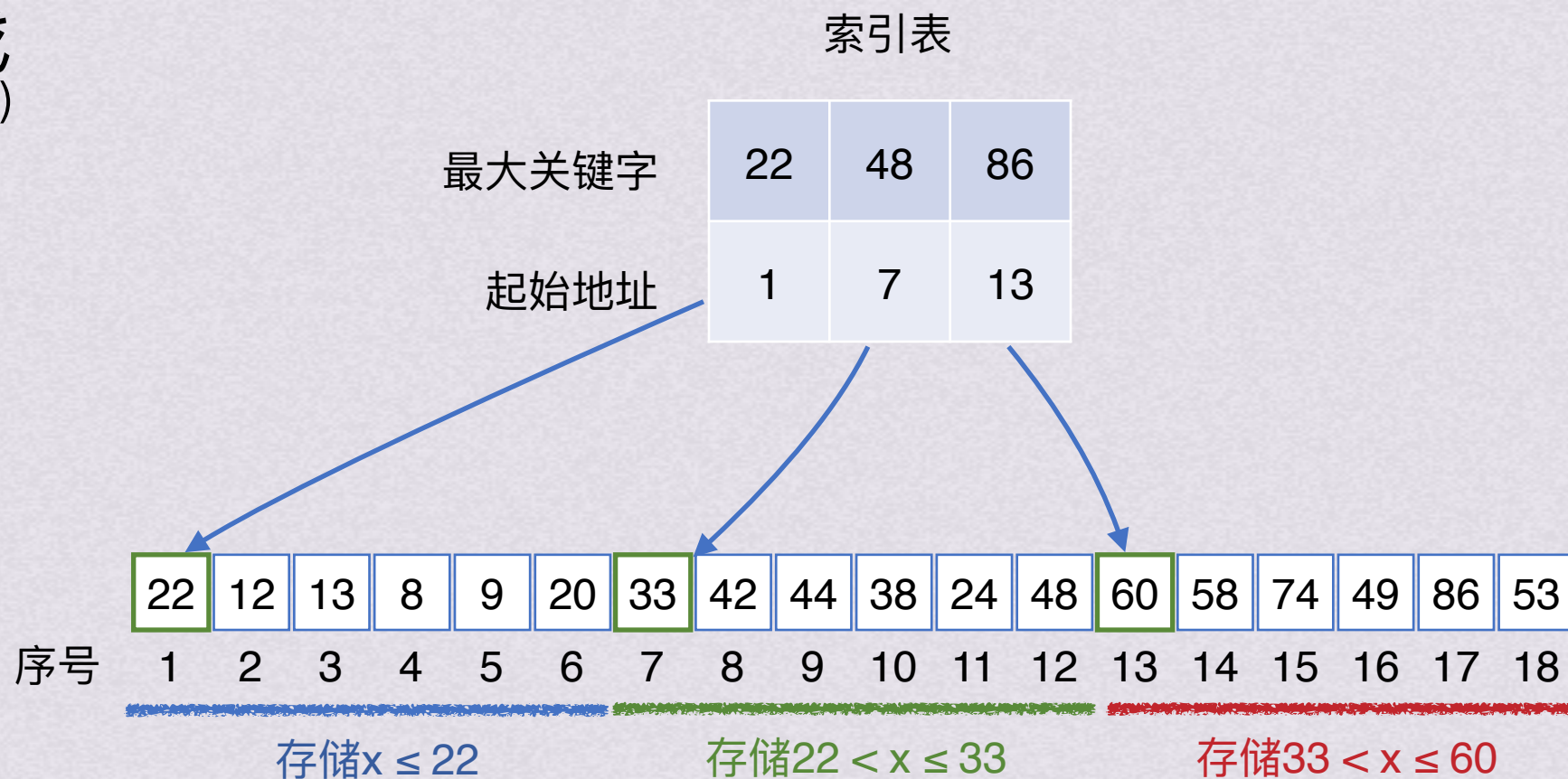
L_w : 在块中查找元素的平均查找长度

每块查找的概率为 $1/b$, 块中每个记录的查找概率为 $1/s$

长度为 n 的表均匀地分成 b 块, 每块含有 s 个记录, 即 $b = \lceil n/s \rceil$



分块查找 (索引顺序查找)



分块查找的平均查找长度为: $ASL_{bs} = L_b + L_w$

$$= \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2} \left(\frac{n}{s} + s \right) + 1$$

查找索引表的过程还可优化为折半查找 $ASL'_{bs} \approx \log_2 \left(\frac{n}{s} + 1 \right) + \frac{s}{2}$